PSynergy

An Independent Journal for PlayStation 2 Linux Developers and Enthusiasts

Issue 2, March 18, 2004.

Table of Contents

EDITORIAL **2**

THIS MONTH IN THE FORUMS **3**

PROJECT SPOTLIGHT **4**

FEATURES **5**

FEATURE: HIGH RESOLUTION TIMING IN PS2 LINUX 5
Understanding the Timers *7*
How to use CPerfTimer *9*
FEATURE: THE PS2 DIRECT MEMORY ACCESS CONTROLLER 10
Function of the DMAC *10*
Normal Mode DMAC Transfer *11*
Source Chain Mode Transfer *11*
Conclusions *13*

PS2 DEVELOPMENT TIPS **14**

FEEDBACK AND NEXT MONTH IN PSYNERGY **17**

End-Reader License

You my freely reproduce and redistribute this publication **in its entirety** through any means.

You may freely redistribute any article contained herein **in its entirety** as long as the name of the original author and the text "as per The Psynergy Journal" are placed both at the beginning and the end of the article. Also, this license paragraph must be copied and appended to the article to ensure that it continues to apply to the redistributed article.

Article Submission Guidelines

You may submit a copy of your article manuscript to patryk@pakl.net. For your article to even be considered, it must be related to development for the Playstation 2 Linux kit. Please note that this includes programming for the RTE, so long as anyone with the Kit can learn from/use it. All submissions will be edited for content, clarity and brevity and the author is given at least 3 days to approve final copy. A suggested template for articles will be provided at http://psynergy.pakl.net/ template.html. Finally, by submitting your article, you agree that it can be redistributed freely, only in its entirety, as long as your name is placed at the beginning and the end of it.

"Sony" is a registered trademark of Sony, Inc. "PlayStation", "PlayStation2", the PlayStation "PS" logo, and all associated logos, are trademarks of Sony Computer Entertainment, Inc. "PSynergy" and is NOT associated in ANY way with Sony Computer Entertainment, Inc.

Editorial

Hello,

Aside from a few typos in the first issue we got off to a fairly good start last month. Thanks for all the great feedback – the more feedback, the better.

This issue is the first one to make use of the PSynergy Snippets system. Next to each piece of code of substantial size, you will find a snippet URL – type this into your browser and you will have syntax-highlighted, copy-and-pastable access to the code in the article.

We need article submissions!! Send them in, either short or long. Also it is helpful if you can send drafts and/or titles early.

Patryk Laurent
Editor-in-Chief, PSynergy
patryk@playstation2-linux.com

This Month in the Forums

A Monthly Column in Psynergy

Playstation 2 Linux kit price drops to $99!
http://playstation2-linux.com/forum/?group_id=1

It was announced a few days ago that the price of the kit has dropped reflecting the decreasing cost of the hardware. However, once the kits have been sold, there are apparaently no plans to make any more. Grab'em while you can!

Object Rendering Hierarchy Design Discussion
http://playstation2-linux.com/forum/message.php?msg_id=41785

Check out the Developers forum for an interesting discussion between PS2 Linux developers on how to port an abstract game framework to the PS2.

GS Tag Special -- Triangle Strips, and the ADC Flag
http://playstation2-linux.com/forum/message.php?msg_id=41888

Extensive discussion on the ADC flag, and a discussion of RET and CALL tags.

Project Spotlight

Projects: 1) PS2 Nohdd Updates; 2) sps2dev/mod Updated to 0.4.0; 3) "Coding for the Playstation 2" Introduction

1) PS2 Nohdd Updates

Michael Mahr's fantastic work on boot images for PS2 Linux without a hard disk are now available through the nohdd project at http://playstation2-linux.com/projects/nohdd. For those of you who aren't familiar with PS2NOHDD yet, you can go to your friend's PS2 and boot up your games, apps and demos with just DVD #1 and your memory card.

2) sps2dev/mod 0.4.0 Update!!!!

Sauce (Steven Osman) posts:
"This version of sps2dev contains a number of updates primarily geared towards making a developer's life a bit easier. In addition, there are a few features that help developers fine-tune the performance of their application."

Also – *make sure to check* in the docs/ directory of the sps2dev! There is documentation there (contributed by Henry Fortuna) for the two tutorials.

3) Coding for the PlayStation® 2 Introduction
http://playstation2-linux.com/coding-on-playstation2.php

Sarah Ewen posts:
"Have you thought about trying to program your PlayStation 2's hardware using your Linux kit, but weren't sure where to get started? There are a few different approaches you can take to getting started, and a growing number of tools, utilities and tutorials to help you get underway. I've summarised as many as I can in this new guide. I hope you find it interesting and useful - your feedback is very welcome, and I'll add in any extra suggestions over time."

Features

Feature: High Resolution Timing in PS2 Linux

Jonathan Hobson jonathan@jhobson.co.uk.

Developing for a console we quite often have the luxury of having our software always run at a fixed frame-rate. This makes things easy for us when coding animation or other time dependent systems. The catch is that for this to work our games must be able to sustain 60fps (30fps for some games) at all times. This becomes difficult as you start doing more and more work per frame. To help you do achieve a constant frame rate, you really need to be able to profile your code, and for that you need a high resolution timer.

Some of you may have played around with the four EE timers (Tn_COUNT etc) only to find they aren't that accurate without using interrupts (which we can't do under PS2 Linux). If you have checked COREUM_E.pdf you probably found the performance timers which are exactly what we want! Unfortunately COP0 is restricted to kernel mode users only. Even with SPS2 we can't get access to it in our user mode applications.

Fortunately the people who wrote the PS2 Linux kernel thought to give us some ioctls to use for accessing the performance counters. If you have seen the documentation for these ioctls inside /usr/doc/Playstation2 then you will know that it is fairly cryptic. The performance counters are actually very simple and easy to use. The aim of this tutorial is to explain in plain English how to use the performance counters in your own code.

Before we start you might want to take at least a brief look at chapter 7 of COREUM_E.pdf. This will tell you a little about the performance counter registers and what can be measured with them. In this tutorial we will only be measuring CPU cycles which then by dividing by the CPU speed will give us a very accurate approximation of the elapsed time.

I have constructed an easy to use performance timer class that you are free to use as you like:

perftimer.h (http://psynergy.pakl.net/snippets/?id=14)

```
#ifndef __PERFTIMER_H__
#define __PERFTIMER_H__

#include <linux/perf_dev.h
#define CPU_APPROX_HZ 293270000.0f
#define PERFTIMER CPerfTimer::GetPerfTimer()

class CPerfTimer
{                                // continues on next page
```

```
                                    // continued from previous page
public:
      void SetEvents(const int iEvent1, const int iEvent2)
      {
            m_pdcNew.ctr0_target = CCR_TARGET_ALL;
            m_pdcNew.ctr0_event = iEvent1;
            m_pdcNew.ctr0_value = 0;

            m_pdcNew.ctr1_target = CCR_TARGET_ALL;
            m_pdcNew.ctr1_event = iEvent2;
            m_pdcNew.ctr1_value = 0;

            m_pdsc.old_counters = 0;
            m_pdsc.new_counters = &m_pdcNew;

            ioctl(m_fdPerfCounter, PERF_IOCSETCOUNTERS, &m_pdsc);

            m_pdsc.old_counters = &m_pdcCurrent;
            m_pdsc.new_counters = 0;
      }

      void Poll()
      {
            ioctl(m_fdPerfCounter, PERF_IOCSETCOUNTERS, &m_pdsc);
      }

      const __u64 & GetCTR0Value()
      {
            return m_pdcCurrent.ctr0_value;
      }

      const __u64 & GetCTR1Value()
      {
            return m_pdcCurrent.ctr1_value;
      }

      static CPerfTimer & GetPerfTimer()
      {
            return PerformanceTimer;
      }

      ~CPerfTimer();
      static CPerfTimer PerformanceTimer;

private:
      CPerfTimer();

      int m_fdPerfCounter;
      perf_dev_set_counters m_pdsc;
      perf_dev_counters m_pdcNew, m_pdcCurrent;
};

#endif
```

perftimer.cpp (http://psynergy.pakl.net/snippets/?id=15)

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/ps2/dev.h>
#include "perftimer.h"

CPerfTimer CPerfTimer::PerformanceTimer;

CPerfTimer::CPerfTimer()
{
	m_fdPerfCounter = open ("/dev/perf_counter", O_RDWR);

	if(m_fdPerfCounter <= 0)
	{
		printf("Couldn't open /dev/perf_counter. CPerfTimer will
not work.");
	}

	ioctl(m_fdPerfCounter, PERF_IOCSTART, NULL);

	SetEvents(CCR_CTR0_CPU_CYCLE, CCR_CTR1_CPU_CYCLE);
}
```

Understanding the Timers

If you just want to use the timers and don't care how they work you can skip this section and go straight to "How to use CPerfTimer". If you want to understand how the timers work and perhaps dabble in some of their advanced features then this section is for you.

There is no better place to start when understanding a class than the constructor. As you can see the first thing that is done is to open /dev/perf_counter in read-write mode. This returns a standard Linux file descriptor that is then checked to make sure that it is valid. The only reason it would ever be invalid is if another application was currently using the performance counters. Next we call the PERF_IOCSTART ioctl that initiates the link between the performance counters and the kernel. Finally the SetEvents function is called initialising both timer events to CCR_CTRn_CPU_CYCLE.

The SetEvents function is the most important function in this class. Before I explain what it does, I should first explain a little more about the performance counters. On the PS2 there are two performance counters: CTR0 and CTR1. Each performance counter has three values: target, event, and value. Target and event both map to the PCCR (performance counter control register: P142 COREUM_E.pdf). "Event" tells the counter which event should be counted. "Target" says in which PS2 operation modes the event should be counted. Finally "Value" as you have probably guessed is the counter's value.

Each CTR can count one of 16 different events. The events differ slightly for each counter although some are the same. You can find all of the different events in /usr/src/linux-2.2.1_ps2/include/asm-mips/r5900_perf_counter.h. You should note however that due to the Linux kernel running in the background almost all of the very specific counters (such as cache hits and misses) will be tainted and you will find it hard to get counts for just your program alone.

The target value allows us to specify which areas of operation should trigger the timer. The choices are CCR_TARGET_EXL (operation in level 1 exception handler), CCR_TARGET_K (operation in kernel mode), CCR_TARGET_S (operation in supervisor mode), and finally CCR_TARGET_U (operation in user mode). In the sample code the target is always set to CCR_TARGET_ALL so that the events are counted in all operation modes. This is because we want to use the CPU cycles for timing and therefore don't want to miss any events. If you can be certain that your application is the only thing running in user mode (I'm not sure if this is possible) then the performance timers will become far more useful as you will be able to measure the very specific performance data. For instance you could measure all of your programs instruction cache misses by setting target to CCR_TARGET_U and changing the event to one such as CCR_CTR0_ICACHE_MISS.

The final thing that needs to be explained is how the PERF_IOC_SETCOUNTERS ioctl works. Inside r5900_perf_dev.h (same path as r5900_perf_counter) are two structures: perf_dev_counters and perf_dev_set_counters.
The perf_dev_counters class has six variables contained within it: target, event, and value for both CTR0 and CTR1. The perf_dev_set_counters has two perf_dev_counters pointers inside it. You pass the perf_dev_set_counters structure into the ioctl which performs the following pseudo-code:

```
if(pdsc.new_counters != NULL)
{
      Write all six values from pdsc.new_counters to the corresponding
      hardware performance counter registers.
}

if(pdsc.old_counters != NULL)
{
      Write all data from the hardware performance counter registers
      into all six members of pdsc.old_counters.
}
```

As you can see, to write to the performance counters you set new_counters to point to a perf_dev_counters structure filled with the values you want written to the hardware. To read from the performance counters you set old_counters to the address of a valid perf_dev_counters structure. The performance counter values are then written to this structure.

In general when you are writing to the performance counters you set

old_counters to null unless you want to store the old values. When reading you will want new_counters set to null so that the performance counters are not written to.

How to use CPerfTimer

I constructed CPerfTimer as a singleton. If you are not familiar with the singleton pattern all it means is that CPerfTimer acts like a global instance without needing any messy global variables. The constructor is called automatically when the static member variable PerformanceTimer is created. This means that without you having to write any code at all the class is all set up and ready to run. Notice that the constructor is private so you cannot create other CPerfTimer instances.

The following code demonstrates how you might use the timer class in your own code:

```
PERFTIMER.Poll(); // Update the performance timer values
uint64 t0 = PERFTIMER.GetCTR0Value();

< Insert code to be timed here >

PERFTIMER.Poll();
Uint64 t1 = PERFTIMER. GetCTR0Value();
float fTimeElapsed = (float)(t1 - t0) / CPU_APPROX_HZ;
```

Poll() reads the times from the performance counters. GetCTR0Value returns the value that was just read from CTR0. The values returned are in CPU cycles, therefore to get the time elapsed in seconds the number of cycles elapsed between calls to Poll() is divided by the approximate CPU Hz. You may want to use CPU KHz instead to give the value in milliseconds instead which would probably give better accuracy.

A quick note about the CPU Hz value: This value was taken achieved by experimentation and is slightly less than the stated value for the EE. You might want to take this value with a pinch of salt, but I find it works perfectly for me.

If you have any questions or comments about this article feel free to email me at jonathan@jhobson.co.uk. I can also usually be found in the #sps2 IRC channel on irc.freenode.net.

Features

Feature: The PS2 Direct Memory Access Controller

Dr Henry S Fortuna (h.s.fortuna@abertay.ac.uk)
University of Abertay Dundee, Scotland UK

Introduction

The Direct Memory Access Controller (DMAC) is a vital component for maximising the performance of the PS2. In this article some of the basic features of the DMAC will be describes. This article is only meant to provide a flavour of the operation of the DMAC and those interested in further details are directed towards Chapter 5 of the EE User's Manual.

Function of the DMAC

The DMAC is responsible for intelligently transferring data between main memory and the peripheral processors and between main memory and scratchpad RAM. During DMAC transfers, the main CPU is freed from involvement, thus speeding up overall system operation. Data, which must be aligned on a qword (128 bit) boundary, is transferred in qword units. There are 10 DMAC channels that are allocated to various destinations within the PS2. Channels that are commonly used when creating applications under PS2 Linux are channel 0 to VIF0, channel 1 to/from VIF1, channel 2 to GIF and channels 8/9 from/to scratchpad RAM.

In terms of performance, the DMAC can transfer data over the main bus at a maximum rate of 2.4 Gbytes/sec. This can be compared with Intel's Accelerated Graphics Port (AGP) x4 bus that has a bus bandwidth of 1.1 Gbytes/sec and the AGPx8 bus with a transfer rate of 2.1 Gbytes/sec.

Transfer addresses given to the DMAC must be specified in terms of physical memory addresses. Normal program variables and dynamically allocated memory in an operating system such as Linux use virtual addresses which are constant for a given program, but the physical address of these variables and memory can change as the operating system pages them in, out and around physical memory. This leads to one of the main purposes of the Direct PS2 Access Environment (SPS2) module (which is required in order to directly access the DMAC under PS2Linux), which is the allocation of un-swappable physical memory that is guaranteed to have a constant physical address. In essence this feature of SPS2 allows the programmer to allocate un-swappable memory that can then be used by the DMAC to transfer data to/from the various processors and scratchpad RAM.

There are 4 different modes of DMAC transfer: Normal, source chain, destination chain, and interleave. In this article only the normal and source chain transfer modes will be discussed.

The DMAC transfer process is controlled by a number of 32-bit registers within the EE core. Some of these registers have multiple values packed together into bits fields. The channel control register (CHCR), the memory address register (MADR), and the quad word count register (QWC) are needed to set up a normal mode DMAC transfer. The channel control register (CHCR), the tag address register (TADR), and the quad word count register (QWC) are needed to set up a chain mode DMAC transfer. SPS2 has predefined unions and structures which can be used to configure these registers and there are pointers defined in order to directly access these Emotion Engine registers.

Normal Mode DMAC Transfer

Normal mode transfer moves a continuous section of data between main memory to one of the other processors or scratchpad RAM. All that is required to set up the transfer is to configure the three registers: QWC, MADR and CHCR. An instance of these registers exists for each of the DMAC channels and they are identified by prefixing the register name with the channel number, e.g. D0_CHCR refers to the channel control register for DMAC channel 0 to VIF0.

The QWC register specified the number of qwords to be transferred and the MADR register specifies the start address in memory of the data to be transferred. The CHCR register has a number of fields but only two of them, the mode (MOD) and the start (STR) are needed for normal mode transfers. The MOD field tells the DMAC what mode of transfer is required, which in this case will be normal mode. When the STR bit of the CHCR register is set to one the DMAC transfer is started.

It is possible to determine when the DMAC transfer has finished by sampling the STR bit of the CHCR register; this bit is reset to zero when the transfer completes.

An example of using this DMAC in normal mode can be seen in the tutorial entitled “Using The DMAC” which can be found at http://www.hsfortuna.pwp.blueyonder.co.uk/.

Source Chain Mode Transfer

Source chain mode allows data to be sent from multiple places in memory to one destination using just one transfer initiation. This is achieved by feeding the DMAC commands, called DMA tags. These DMA tags are used to find both the next chunk of data to send and the next DMA tag if there is one.

DMA tags are 128 bits long, but only the lower 64 bits are used. Anything can be placed in the top 64 bits of the DMA tag and this is often taken advantage of to insert VIF codes for example.

The combination of the actual stream of data and DMA tags is called a DMA chain. A DMA chain can be thought of as being made up of links, with each link having one DMA tag and associated data.

The QWC field of the DMA tag specifies how much data is in the link,

measured in quad words. Each tag can have a different type, which is set in the identification (ID) field. The tag's ID affects what value is put into the ADDR field of the tag.

DMA tags that are commonly used are cnt, next, ref, refe, and end. There are also refs, call and ret tags but these are not described in this article. A short explanation of each of these tags is given below.

cnt: This tag says to transfer QWC of data following the tag, and then read the quad word after that data as the next DMA tag.

next: This tag says to transfer QWC of data following the tag, and then read the next tag from the address in the ADDR field of the tag.

ref: This tag says to transfer QWC of data from the address in the ADDR field of the tag then read the quad word following this tag as the next tag.

refe: This provides the same function as the ref tag except that the transfer ends after transferring the data from the address in the ADDR field.

end: This tag says to transfer QWC of data following the tag, and then end the transfer.

These tags can be seen in operation in the tutorial entitled "Using DMAC Tags" which can be found at http://www.hsfortuna.pwp.blueyonder.co.uk/.

In order to help understand the use of DMAC tags and source chain mode transfers consider the table below which shows some data and DMA tags laid out in memory. Note that the DMA tags and data are scattered around memory.

Address	Contents
0x0000	DMA Tag #1: ID = NEXT, ADDR = 0x0030, QWC = 2
0x0010	String 1
0x0030	DMA Tag #2: ID = REF, ADDR = 0x1000, QWC = 2
0x0040	DMA Tag #3: ID = CNT, ADDR = 0, QWC = 2
0x0050	String 3
0x0070	DMA Tag #4: ID = END, ADDR = 0, QWC = 2
0x0080	String 4
0x1000	String 2

The DMAC is initiated and reads DMA Tag #1 from address 0x0000. It will transfer the 2 quad words of string 1 and then read in DMA Tag #2 from the address specified in the ADDR field of DMA tag #1. DMA Tag #2 causes the DMAC to transfer the two quad words from the address in the ADDR field of DMA Tag #2, which is String 2, and then read in DMA Tag #3. DMA Tag #3 tells the DMAC to transfer the following two quad words, which is String 3, then read the quad word after that as the next tag which is DMA Tag #4. The DMAC then transfers the final two quad words (string 4) and ends the transfer. All the

strings are transferred in the order that is specified (strings 1 through to 4) even though they are dotted around in memory.

Conclusions

This short article has illustrated some of the features of the DMAC within the PS2. It is worth mentioning that memory allocated under SPS2 is only physically continuous in 4k blocks but the source chain mode feature of the DMAC can be used to stitch these blocks together so that large chunks of data can be transferred around the system under PS2 Linux.

Dr Henry S Fortuna
Lecturer in Computer Games Technology
University of Abertay Dundee
Scotland UK

h.s.fortuna@abertay.ac.uk

18 March 2004

PS2 Development Tips

Tips: 1) How to tell when VU1 has Finished; 2) DMA Transfer Maximum; 3) Scratchpad Usage

Tip #1: How to tell when VU1 has finished

http://playstation2-linux.com/forum/message.php?msg_id=38026

bjt: "Use a FLUSHE vifcode to cause the DMA transfer to stall until the VU has finished. Poll STR to check whether the transfer has completed.

If you poll STR in a tight loop, the increased demands on main bus access will slow your DMA transfer down. Poll STR less frequently, or use sps2WaitForDMA (if you are using sps2)."

Hikey : "To add some info on what bjt already said:

Build your double buffered DMA packet like this (only an example):
- CALL tag to microcode to send it each time.
- REF tag to matrices, tex0 settings, texflush, etc...
- MSCALF vifcode
- REF tag to data
- MSCNT vifcode
- REF tag to more data
...
- last MSCNT vifcode
- FLUSHE vifcode

10 Build current DMA packet
20 Wait for VSync
30 Swap display buffer
40 Wait for DMA transfer completion
50 Send current DMA buffer
60 Swap DMA buffer
70 Goto 10"

Sparky : "look up vif1_stat in the vif section of the EE Users manual. It has a status bit for it.

In general you don't need to wait for vu1 on the main cpu side though. You just double/tripple buffer and do mscalls after upload (dma+vif). The vif will automatically

stall on the mscall if the vu1 is busy. If you're single buffering you'll need to use one of the several flush vif instructions."

And http://playstation2-linux.com/forum/message.php?msg_id=37445
(In reference to blevine's question)
Sauce: "Your benchmark is certainly valid, but unfortunately you don't seem to be making the best of sps2.

The reason you're getting poor performance is because you're invoking sps2WaitForDMA four times for each transfer. sps2WaitForDMA waits "nicely" for the DMA to complete. By nicely, I mean that it yields control of the processor to other programs in the system. While this is a good thing to do if you need to wait for the completion of a massive transfer that you just started, it is not a good thing to do for small transfers that you do often. For those cases, I recommend a simple greedy spinlock. Basically, a loop that checks the STR bit on CHCR to make sure the transfer is still in progress.

Try replacing your waits with:

```
while ((*EE_D1_CHCR) & 0x100);
```

This will loop so long as the STR bit is set.

I say it's greedy because it doesn't yield to other programs automatically, on the other hand, if your concern is a high performance application, then you don't want users to be running 50 million programs at the same time anyway.

Now... That being said... The second problem with your approach is that it doesn't take full advantage of what the DMAC can do for you, even when using SPS2. I strongly suggest you do a teenie bit of reading into source chain mode. Basically, you can load up the entire set of data in one transfer by using a chain. The chain could consist of ref tags which basically say (copy x bytes of data from location y).

Once you've updated your code to do a chained transfer, you'll need only one spinlock for the whole transfer. Try implementing that and rerun your benchmarks. You should get much better results. "

Tip #2: DMA Transfer Maximum

http://playstation2-linux.com/forum/message.php?msg_id=41769

amuehlem: "I'm trying to transfer a picture from memory to the GS via path 2. The picture contains only points, each with RGBA and XYZ values.

I can't send the complete picture (eg. 640 x 480 points) using only one DMA transfer, sending only a part of the picture is ok. Is there a maximum size a DMA transfer can have? Is it not possible to transfer a complete (640x480) picture to the GS?

Thank you for your help
Andreas"

Hikey "Hi Andreas,

The limit is 1MB, so no you cannot send a 640x480 picture in one go in normal

mode. You can chain several 1MB transfers though, so a single chain will do.

You are sending a picture where every point is a vertex ? This is highly inefficient, so unless you are investigating a technique that requires this, you should really send the picture as a texture. If you are doing a "dotted flag" kind of effect, then fair enough, but if it's a normal picture where all the pixels are touching each other it's not the way to go. HTH."

Tip #3: ScratchPad Usage

http://playstation2-linux.com/forum/message.php?msg_id=39637

antigrav: "I'm trying to understand how to properly use the ScratchPad memory. When DMA is active, from what I understand, using the SP is advantageous because it keeps you off the data bus which is being utilized by the DMA right? Well my question is from the point of view of a C programmer. What memory am I allowed to access and not access besides the SP area mapped by sps2? I assume the stack is alright. I mean in a for loop such as "for(i=0;i<n;i++)", I don't need to put the 'i' and 'n' variables in the SP area too do I? What type of memory/variables can't I access? I don't know how the instruction and data caches are viewed from a C program. Is the stack located in the data cache? If someone could clear this up a bit for me, I'd be very grateful. Thanks!"

Hikey: "Indeed, using the SPR is usually a good idea. The stack is in the data cache by default, where it really belongs, so accessing the stack should not be a major hit (although that depends a lot on the code).

Usually though, you want to think of it the other way around. You want to design your DMA packets so that the tranfers don't saturate the bus, to leave some badwidth to the CPU to load its data.

For example, your VIF chain should be packed so that transforming the geometry takes longer than transfering the data to the VIF. That way, the DMA transfer will stall on MSCNT and the bus will be free for the other transfers to go on. Also, that means that if the VIF transfer is slowed down by the CPU accessing the bus or by a texture transfer (which are the two biggest bandwidth eaters), the VU will not starve on data.

To summarise: use SPR (possibly in double buffers) to access small data packets that you read and write. Use prefetch to load the data in cache before you need it. Use uncached accelerated to write DMA packets (for example). Keep the DMA transfers small (pack your VIF data, use smallish textures).

As always, it's easier said than done. "

Feedback and Next Month in PSynergy

Feedback

I just want to thank you for putting the efforts into starting this e-mag for PS2 Linux users. It's a great idea. Now I feel inspired to actually take my PS2 Linux kit out of closet and hook it up for some experiments. Your work and work of all the people who helped you is reaching masses and is much appreciated.

What I do consider important is a formal anouncement on title playstation2-linux.com page. That will bring more casual ps2 linux visitors to the mag. I only knew about PSynergy mock issue because I did a pretty serious research a few months ago, going through almost every project on the site. And yesterday I decided to check if there is any progress. A title page and maybe an article on slashdot would defenitivly help bring more people in :)

Thank you,
Adrian (dhau@playstation2-linux.com)

Adrian – thanks for your feedback. It's great to know that Psynergy is helping to motivate PS2 Linux Developers.

Look for PSynergy **Next Month!**